

实验记录

实验步骤

1. 启动 c8.exe
2. 启动 Camunda
3. Camunda 部署一个实例,此时可以在 Operate 上看到一个卡住的进程
4. 启动 index.ts(指令:npx ts-node index.ts)
5. Postman 发送 Transport 与 FreightForwarder 下面列出的 JSON 消息(也可使用 sendMsg.ts / sendFFMsg.ts 在新终端执行代替 Postman)
6. 可以观察到 Operate 上进程继续进行,直至两个流程都到达 Completed

Transport

接口名

```
POST http://localhost:8080/v2/messages/publication
```

Message 1 — FF Equipment Receipt 送达 Transport

对应 Transport 流程中 "FF Equipment Receipt received" 中间消息事件,由货代下发设备交接单。

```
{
  "name": "Message_FF_Equipment_Receipt_received",
  "correlationKey": "ORDER-20260420-001",
  "timeToLive": 300000,
  "variables": {
    "receiptId": "EIR-MSK-992384",
    "pickupDepot": "DEPOT-BAOSHAN-01"
  }
}
```

Message 2 — 空箱(Empty CTN)送达 Transport

对应 Transport 流程中 "Empty CTN received" 中间消息事件,由堆场返回空箱。

```
{
  "name": "Message_Transport_empty_CTN_received",
  "correlationKey": "ORDER-20260420-001",
  "timeToLive": 300000,
  "variables": {
    "ctnNumber": "CTN-884821"
  }
}
```

Message 3 — 出口重箱送达 Transport

对应 Transport 流程中 "Outbound CTN received" 中间消息事件,由货主交付出口重箱。

```
{
  "name": "Message_Owner_Outbound_CTN_received",
  "correlationKey": "ORDER-20260420-001",
  "timeToLive": 300000,
  "variables": {
    "ctnNumber": "CTN-884821"
  }
}
```

实验结果

Transport 流程在 Operate 中的执行结果

```

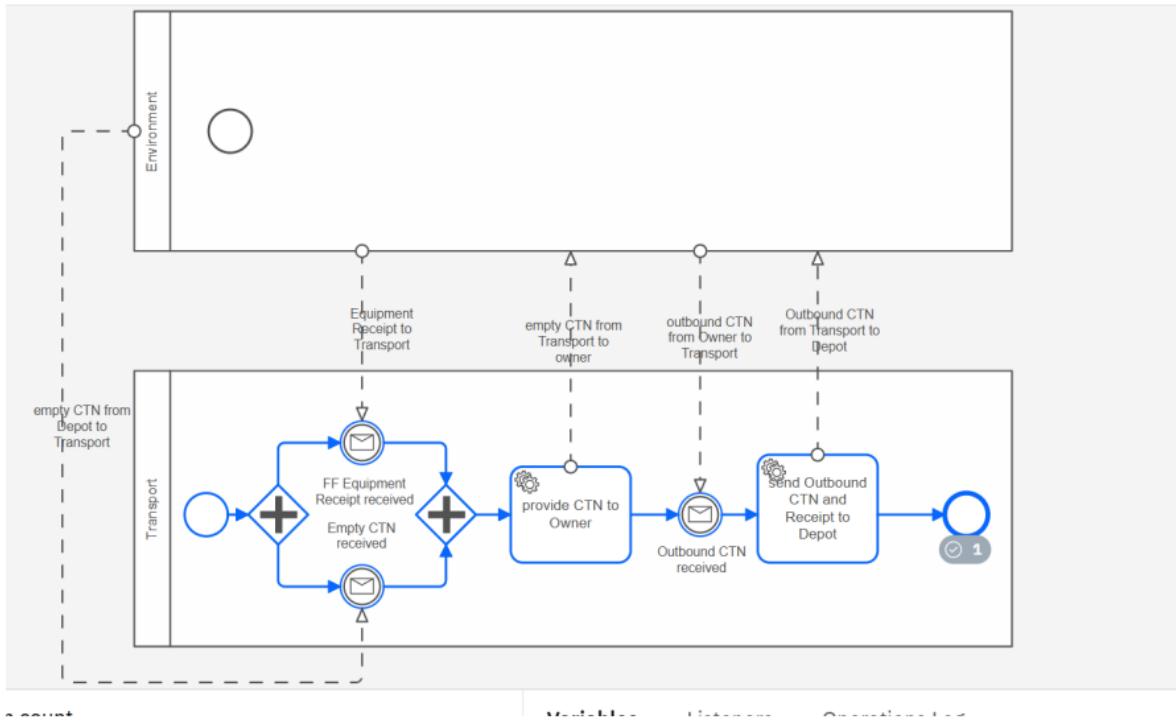
C:\Users\35225\Desktop\camunda8-getting-started-bundle-8.9.0-alpha4-windows-x86_64\camunda-8-get-started\nodejs\source>npm start

> nodejs@1.0.0 start
> npx ts-node ./source/index.ts

Job workers started. Waiting for jobs...

info: [派箱任务] 开始为订单 [ORDER-20260420-001] 分配空集装箱并安排派车... {"timestamp":"2026-04-27T05:09:23.810Z"}
info: [派箱任务] 成功分配。车牌：沪A-12345，箱号：CTN-563810 {"timestamp":"2026-04-27T05:09:25.314Z"}
info: [重箱进港] 准备将集装箱 [CTN-884821] 送达堆场，关联设备单：[EIR-MSK-992384]... {"timestamp":"2026-04-27T05:14:45.979Z"}
info: [重箱进港] 任务完成：2251799814217556 {"timestamp":"2026-04-27T05:14:47.982Z"}

```



FreightForwarder

流程介绍

FreightForwarder 是整个航运订单流程的核心协调方,本实验中其流程定义来自 FreightForward.bpmn(Process_0jeubsp)。

FF 流程的启动方式是 Message Start Event:监听 Message_Owner_order_received 消息,一旦收到该消息,Camunda 会自动新建一个 FF 流程实例,并将消息中的变量(如 orderId)绑定为流程的 correlation 上下文。

流程开始后会进入并行网关(Parallel Gateway),分两条分支同时执行两个 ServiceTask:一是 send S/O to SA(向船代发送订舱单,worker type:so-to-sa),二是 send order info to Customs Broker(向报关行发送订单与箱单发票信息,worker type:order-info-to-cb)。两个

分支都完成后,流程在汇合网关重新汇聚。

随后流程依次等待两个中间消息事件:Manifest received(等待船代回传的舱单 Message_FF_Manifest_received)和 SA Equipment Receipt received(等待船代下发的设备交接单 Message_SA_Equipment_Receipt_received)。两条消息都到齐之后,流程执行最后一个 ServiceTask —— send Equipment Receipt to Transport(将 EIR 转发给车队,worker type:equipment-receipt-to-transport),完成后到达结束事件,整个 FF 流程实例 Completed。

FF 流程的全部三个消息接收点都使用 correlationKey =orderId 进行匹配,因此发送消息时必须保证 correlationKey 字段值与流程实例上下文中的 orderId 一致。

接口名

```
POST http://localhost:8080/v2/messages/publication
```

Message 1 — Owner Order 送达 FF(启动 FF 流程实例)

对应 FF 流程的 Message Start Event "order received"。该消息一发出,Camunda 即新建一个 FF 流程实例并将变量绑定到上下文,后续 SA 与 CB 两个分支并行启动。

```
{
  "name": "Message_Owner_order_received",
  "correlationKey": "ORDER-20260420-001",
  "timeToLive": 300000,
  "variables": {
    "orderId": "ORDER-20260420-001",
    "shipperName": "ACME Trading Co., Ltd.",
    "consignee": "Global Buyer Inc.",
    "pol": "CNSHA",
    "pod": "USLAX",
    "commodity": "Electronics",
    "quantity": 1,
    "ctnType": "40HQ"
  }
}
```

Message 2 — Manifest 送达 FF

对应 FF 流程中 "Manifest received" 中间消息事件。FF 在两个并行 ServiceTask 完成后会停在此节点等待,直到收到由船代回传的舱单消息后继续。

```
{
  "name": "Message_FF_Manifest_received",
  "correlationKey": "ORDER-20260420-001",
  "timeToLive": 300000,
  "variables": {
    "manifestId": "MNF-FF-20260420-001",
    "shipName": "MSC OSCAR",
    "voyageNumber": "V0428E",
    "issuedBy": "SA-MAERSK-01"
  }
}
```

Message 3 — SA Equipment Receipt 送达 FF

对应 FF 流程中 "SA Equipment Receipt received" 中间消息事件。该消息到达后,FF 才会执行最后一个 ServiceTask "send Equipment Receipt to Transport"(也就是把 EIR 转发给车队),整个 FF 流程随即 Completed。

```
{
  "name": "Message_SA_Equipment_Receipt_received",
  "correlationKey": "ORDER-20260420-001",
  "timeToLive": 300000,
  "variables": {
    "receiptId": "EIR-MSK-992384",
    "depotId": "DEPOT-BAOSHAN-01",
    "pickupDepot": "DEPOT-BAOSHAN-01",
    "ctnType": "40HQ",
    "ctnQuantity": 1,
    "validUntil": "2026-04-25T18:00:00Z"
  }
}
```

实验结果

FreightForwarder 流程在 Operate 中的执行结果

在 Camunda Operate 中查看,Freight Forwarder 流程实例从 "order received" 一路推进到结束事件,所有节点全部走过(显示为蓝色),实例计数为 1,说明流程实例已 Completed。

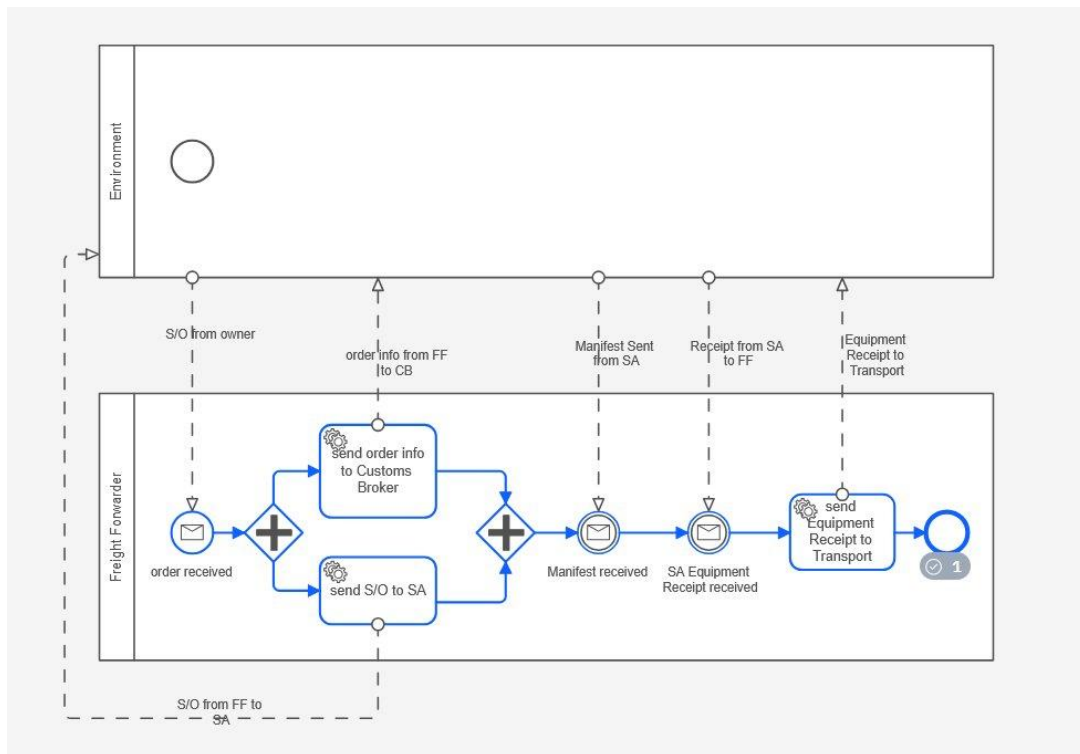


图 1 FreightForwarder 流程在 Operate 上的最终状态(全节点走过,Completed 实例计数 = 1)

FreightForwarder Worker 终端日志

启动 index.ts 后,Transport 与 FreightForwarder 两套 worker 同时连接 Camunda 待命。
当 sendFFMsg.ts 发送三条消息时,worker 终端依次打印 [S/O 分支]、[报关分支] 并行处理日志,以及 [派车节点] 顺序处理日志,与 BPMN 中并行网关 + 顺序节点的结构完全一致。

```

D:\camunda8\ffw-trl-new>npx ts-node index.ts
Job workers started. Waiting for jobs...

- Transport workers: ctn-to-owner, outbound-ctn-to-depot
- FreightForwarder workers: so-to-sa, order-info-to-cb, equipment-receipt-to-transport

info: [S/O 分支] 开始为订单 [ORDER-20260420-001] 向船代发送订舱申请... {"timestamp":"2026-05-07T08:38:02.462Z"}
info: [报关分支] 开始为订单 [ORDER-20260420-001] 向报关行同步订单与箱单发票信息... {"timestamp":"2026-05-07T08:38:02.466Z"}
info: [报关分支] 报关信息同步完成: 2251799813714011 {"timestamp":"2026-05-07T08:38:03.516Z"}
info: [S/O 分支] 订舱单发送完成: 2251799813714012 {"timestamp":"2026-05-07T08:38:03.968Z"}
info: [派车节点] 准备向车队下发提箱指令。订单: [ORDER-20260420-001], 交接单号: [EIR-MSK-992384] {"timestamp":"2026-05-07T08:38:05.536Z"}
info: [派车节点] 设备交接单已成功下发至车队: 2251799813714051 {"timestamp":"2026-05-07T08:38:07.548Z"}

```

图 2 FreightForwarder 三个 ServiceTask 在 worker 终端的执行日志